

Forensics Writeups

Email analysis, PCAP traffic, PowerShell malware and Windows memory.

01 QuackQuackDiriDiriDuck

02 Miletus

03 Dead Weight

01

EMAIL FORENSICS / PCAP / MALWARE TRAFFIC

QuackQuack DiriDiriDuck

The challenge starts with malicious PDF emails, then moves into a PCAP with the ZIP download, DLL request and Qakbot C2 traffic.

EML

tshark

Qakbot

I started from the emails

There was no reason to touch the attachments. I only checked the EML headers and MIME sections, which was enough for the first question.

```
ayham@Ayham: ~/foren... x + v
ayham@Ayham:~/forensics/quack$ unzip -P infected QuackQuackDiriDiriDuck.zip -d work/
Archive:  QuackQuackDiriDiriDuck.zip
  inflating: work/emails/Email1.eml
  inflating: work/emails/Email2.eml
  inflating: work/traffic.pcap
ayham@Ayham:~/forensics/quack$ cd work && grep -RniE "Content-Type: application/pdf|filename=.*pdf" emails/
emails/Email1.eml:18:Content-Type: application/pdf
emails/Email2.eml:21:filename="invoice.pdf"
emails/Email3.eml:24:Content-Type: application/pdf
emails/Email4.eml:19:filename="document.pdf"
emails/Email5.eml:22:Content-Type: application/pdf
ayham@Ayham:~/forensics/quack$ grep -Rli "application/pdf" emails/*.eml | wc -l
5
```

Unpacking the archive and counting the messages with PDF attachments.

What I checked

From, Sender, MIME type, attachment name and the real file extension.

Result

There was 5 emails carrying the malicious PDFs.

ANSWER 1

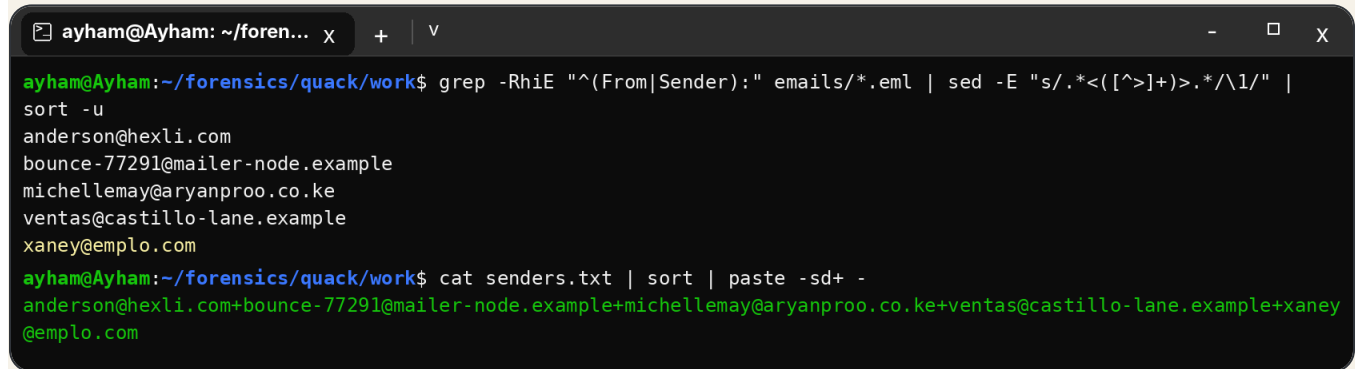
OmniCTF{5}

The emails were normal looking enough, but the headers was doing most of the talking here.

QUACK / SENDER HEADERS

The address that kept breaking my flag

I first collected the `From` values and the answer still failed. Email3 also had a `Sender` header, and that is where **xaney@emplo.com** came from.



```
ayham@Ayham: ~/foren... x + | v
ayham@Ayham:~/forensics/quack/work$ grep -RhiE "^(From|Sender):" emails/*.eml | sed -E "s/.*<([>]+)>.*\/\1/" |
sort -u
anderson@hexli.com
bounce-77291@mailers-node.example
michellemay@aryanproo.co.ke
ventas@castillo-lane.example
xaney@emplo.com
ayham@Ayham:~/forensics/quack/work$ cat senders.txt | sort | paste -sd+ -
anderson@hexli.com+bounce-77291@mailers-node.example+michellemay@aryanproo.co.ke+ventas@castillo-lane.example+xaney@emplo.com
```

Reading both `From` and `Sender`, then sorting the addresses.

Alphabetical order

anderson@hexli.com
bounce-77291@mailers-node.example
michellemay@aryanproo.co.ke
ventas@castillo-lane.example
xaney@emplo.com

ANSWER 2

OmniCTF{anderson@hexli.com+bounce-77291@mailers-node.example+michellemay@aryanproo.co.ke+ventas@castillo-lane.example+xaney@emplo.com}

One missing header was enough to make the whole long flag wrong. That part took longer than it should of.

QUACK / NETWORK TRAFFIC

From the ZIP request to the TLS traffic

The first malicious HTTP request was visible in plain text. After the DLL download, the traffic moved to TLS, so I filtered the ClientHello fingerprint and removed duplicate destinations.

```
ayham@Ayham: ~/foren... x + | v
ayham@Ayham:~/forensics/quack/work$ tshark -r traffic.pcap -Y "http.request" -T fields -e frame.time -e http.host -e http.request.uri
May 24, 2023 16:34:13 adubuildersco.com /ewukhyqpjz/ewukhyqpjz.zip
May 24, 2023 16:35:27 185.81.114.188 /payload.dll
ayham@Ayham:~/forensics/quack/work$ tshark -r traffic.pcap -Y "http.request" -T fields -e http.host -e http.request.uri | head -n 1
adubuildersco.com/ewukhyqpjz/ewukhyqpjz.zip
```

The first HTTP requests that belong to the infection chain.

ANSWER 3

OmniCTF{http://adubuildersco.com/ewukhyqpjz/ewukhyqpjz.zip}

```
ayham@Ayham: ~/foren... x + | v
ayham@Ayham:~/forensics/quack/work$ tshark -r traffic.pcap -Y "tls.handshake.ja3 == 43016d7f7f9336b17c884650d0d2545d" -T fields -e ip.dst -e tcp.dstport | sort -u
142.118.221.248 2222
185.81.114.188 443
188.28.19.84 443
201.130.154.90 443
ayham@Ayham:~/forensics/quack/work$ tshark -r traffic.pcap -Y "tls.handshake.ja3 == 43016d7f7f9336b17c884650d0d2545d" -T fields -e ip.dst -e tcp.dstport | sort -u
| wc -l
4
```

Unique remote IP and port pairs matching the Qakbot JA3 value.

ANSWER 4

OmniCTF{4}

JA3

43016d7f7f9336b17c884650d0d2545d

ANSWER 5

OmniCTF{142.118.221.248:2222+185.81.114.188:443+188.28.19.84:443+201.130.154.90:443}

QUACK / TIMELINE

The times came from the PCAP

I used the packet timestamps, not the email dates. The ZIP request happened first, the DLL followed, then the first matching C2 ClientHello appeared 340.388 seconds later.

```
ayham@Ayham: ~/foren... x + v
ayham@Ayham:~/forensics/quack/work$ tshark -r traffic.pcap -Y "http.request || tls.handshake.type == 1" -T fields
-e frame.time -e http.request.full_uri -e ip.dst | rg
"ewukhyqpjz|payload.dll|142.118.221.248"
2023-05-24 16:34:13 http://adubuildersco.com/ewukhyqpjz/ewukhyqpjz.zip
2023-05-24 16:35:27 http://185.81.114.188/payload.dll
2023-05-24 16:41:07.388 142.118.221.248
ayham@Ayham:~/forensics/quack/work$ python3 -c "print(round((41*60+7.388)-(35*60+27), 3))"
340.388
```

Packet times for the ZIP, DLL and first matching C2 connection.

ZIP GET

2023-05-24 16:34:13

DLL GET

2023-05-24 16:35:27

C2 DELAY

340.388 seconds

6 - ZIP request

OmniCTF{2023-05-24 16:34:13}

7 - DLL request

OmniCTF{2023-05-24 16:35:27}

8 - Delay

OmniCTF{340.388}

9 - Malware family

OmniCTF{Qakbot}

10 - Campaign

OmniCTF{OBAMA264}

Small format issue: OBAMA was not complete. The accepted value was OBAMA264 .

02

STATIC MALWARE ANALYSIS / POWERSHELL

Miletus

A PowerShell sample hidden behind Base64, XOR and gzip. I kept it offline and only copied the decoding logic.

[virus.ps1](#)[XOR](#)[KongTuke](#)

MILETUS / FIRST LOOK

First thing: dont run it

The README said it was real malware, so I treated `virus.ps1` like data. The archive had the warning, the question list and the obfuscated script.

```
ayham@Ayham: ~/foren... x + v
ayham@Ayham:~/forensics/miletus$ unzip -P infected Miletus-player.zip -d work/
Archive:  Miletus-player.zip
  inflating: work/omni/README.txt
  inflating: work/omni/tasks.txt
  inflating: work/omni/virus.ps1
ayham@Ayham:~/forensics/miletus$ cd work/omni && file README.txt tasks.txt virus.ps1
README.txt: ASCII text
tasks.txt:  ASCII text
virus.ps1:  ASCII text, with very long lines
ayham@Ayham:~/forensics/miletus$ sed -n "1,12p" tasks.txt
q1 URL
q2 value appended to HWID
q3 AES padding
q4 crypto implementation
q5 variables in Get-HWID
q6 malware family
```

Basic triage after extracting the password-protected archive.

README.txt

The warning and archive password.

tasks.txt

The six values needed for the final flag.

Nothing was executed. For these questions, static analysis was safer and honestly more useful.

Finding the layers inside virus.ps1

I searched for the operations that rebuild the next stage: Base64 decoding, XOR and gzip. The XOR key was stored directly in the script.

```
ayham@Ayham: ~/foren... x + v
ayham@Ayham:~/forensics/miletus/work/omni$ rg "FromBase64String|GZipStream|-bxor|kzWYK" virus.ps1
[Convert]::FromBase64String($blob)
$key = "kzWYKApomHlfUdaDPT0e"
$out[$i] = $raw[$i] -bxor $xkey[$i % $xkey.Length]
New-Object IO.Compression.GZipStream(...)
ayham@Ayham:~/forensics/miletus/work/omni$ printf "%s\n" "kzWYKApomHlfUdaDPT0e"
kzWYKApomHlfUdaDPT0e
```

The decoding functions and XOR key inside the original script.

LAYER 1

Base64 blob

LAYER 2

XOR with the key

LAYER 3

Base64 and gzip

XOR KEY

kzWYKApomHlfUdaDPT0e

It looked like a giant mess at first, but the actual chain was pretty small once I ignored all the random variable names.

Getting a readable second stage

The local unpacker only repeated the transforms from the PowerShell. After the XOR step, the data decoded into gzip and finally produced `stage2.ps1`.

```
ayham@Ayham: ~/foren... x + v
ayham@Ayham:~/forensics/miletus/work/omni$ python3 unpack.py virus.ps1
stage1.bin written
xor key: kzWYKApomHlfUdaDPT0e
stage2.ps1 written
ayham@Ayham:~/forensics/miletus/work/omni$ file stage1.bin stage2.ps1 && head -n 4 stage2.ps1
stage1.bin: gzip compressed data
stage2.ps1: Unicode text, UTF-8 text
function Get-HWID {
    $computerSystem = Get-CimInstance Win32_ComputerSystem
```

The decoded output and the first readable lines from stage two.

Order

Base64 → XOR → Base64 → gzip → stage2.ps1

Seeing `function Get-HWID` was the point where the ugly part was basically finished.

Most answers was plain text now

I searched the clean stage for URL, password, HWID, AES padding and PBKDF2. Almost the full task list showed up in a few lines.

```
ayham@Ayham: ~/foren... x + v
ayham@Ayham:~/forensics/miletus/work/omni$ rg -n "gloason|16k3g6c0pwnj|SerialNumber|Padding|PBKDF2|Get-HWID"
stage2.ps1
18:$url = "https://gloason.com/white/lrau20bdzx/"
24:$password = "16k3g6c0pwnj"
47:$hwid += $bios.SerialNumber
66:$aes.Padding = "PKCS7"
73:PBKDF2_HMAC-SHA256
91:function Get-HWID {
ayham@Ayham:~/forensics/miletus/work/omni$ sed -n "/function Get-HWID/,/^}/p" stage2.ps1 | rg "^s*\$" | wc -l
8
```

The values used by the second-stage PowerShell.

Network

URL

<https://gloason.com/white/lrau20bdzx/>

Password

16k3g6c0pwnj

Crypto and HWID

Field SerialNumber

Padding PKCS7

Implementation PBKDF2_HMAC-SHA256

Variables 8

The stored password was useful while reading the sample, but it was not part of q1 to q6.

Keeping the task order exactly

The code and behavior matched **KongTuke**. After that I put the six answers in the same order as `tasks.txt` and kept the slash, underscore and hyphen exactly.

```
ayham@Ayham: ~/foren... x + v
ayham@Ayham:~/forensics/miletus/work/omni$ cat answers.txt
https://gloason.com/white/lrau20bdzx/
SerialNumber
PKCS7
PBKDF2_HMAC-SHA256
8
KongTuke
ayham@Ayham:~/forensics/miletus/work/omni$ printf "OmniCTF{%s}\n" "$(paste -sd+ answers.txt)"
OmniCTF{https://gloason.com/white/lrau20bdzx/+SerialNumber+PKCS7+PBKDF2_HMAC-SHA256+8+KongTuke}
```

Joining the six answers in the required q1+q2+... format.

1 - URL

https://gloason.com/white/lrau20bdzx/

2 - HWID field

SerialNumber

3 - Padding

PKCS7

4 - Implementation

PBKDF2_HMAC-SHA256

5 - Variables

8

6 - Family

KongTuke

FINAL FLAG

OmniCTF{https://gloason.com/white/lrau20bdzx/
+SerialNumber+PKCS7+PBKDF2_HMAC-SHA256+8+KongTuke}

03

WINDOWS MEMORY FORENSICS

Dead Weight

The RAM image kept console history, Recycle Bin details, UserAssist entries and the browser trail that pointed to both flags.

Volatility 3

UserAssist

DES

DEAD WEIGHT / VOLATILITY

The console buffer gave me the first lead

I started with `filescan`, `cmdline` and `consoles`. The useful part was the console history because it still had WMIC output and the Recycle Bin commands.

```
ayham@Ayham: ~/foren... x + v
ayham@Ayham:~/forensics/deadweight$ ls -lh DESKTOP-E0CHD27-20250604-195537.raw
-rw-r--r-- 1 ayham ayham 8.0G DESKTOP-E0CHD27-20250604-195537.raw
ayham@Ayham:~/forensics/deadweight$ python3 ~/tools/volatility3/vol.py -f DESKTOP-E0CHD27-20250604-195537.raw
windows.filescan
Volatility 3 Framework 2.x
Progress: 100.00 PDB scanning finished
ayham@Ayham:~/forensics/deadweight$ python3 ~/tools/volatility3/vol.py -f DESKTOP-E0CHD27-20250604-195537.raw
windows.consoles > consoles.txt
```

The first Volatility pass against the memory image.

```
ayham@Ayham: ~/foren... x + v
ayham@Ayham:~/forensics/deadweight$ rg -n "wmic useraccount|Masquerade|\\$Recycle.Bin" consoles.txt
wmic useraccount get name,sid
Masquerade S-1-5-21-3266328033-1872285240-1484667356-1001
C:\\$Recycle.Bin\\S-1-5-21-3266328033-1872285240-1484667356-1001
ayham@Ayham:~/forensics/deadweight$ rg -n "IBZOM62|RBZOM62|Flag.txt" consoles.txt
06/04/2025 10:37 PM 106 $IBZOM62.txt
C:\\Users\\Masquerade\\Downloads\\Flag.txt
del /Q S-1-5-21-3266328033-1872285240-1484667356-1001\\$RBZOM62.txt
```

The Masquerade account, SID folder and deleted Flag.txt entry.

ANSWER 1

OmniCTF{S-1-5-21-3266328033-1872285240-1484667356-1001}

DEAD WEIGHT / DELETED FILE

The Recycle Bin kept the time

The `$I` metadata file stored the original path and deletion time. It showed `$IBZ0M62.txt` at 10:37 PM for the same Masquerade SID.

ANSWER 2

OmniCTF{06/04/2025
10:37}

Original file

C:
\Users\Masquerade\Downloads\Flag.txt

```
ayham@Ayham: ~/foren... x + | v
ayham@Ayham:~/forensics/deadweight$ python3 ~/tools/volatility3/vol.py -f DESKTOP-E0CHD27-20250604-195537.raw
windows.registry.userassist | rg "vrkcybere\Masquerade"
P:\Hfref\Znfdhrenqr\Qrfxgbc\vrkcybere.rkr
ayham@Ayham:~/forensics/deadweight$ python3 -c "import codecs; print(codecs.decode(r"P:\Hfref\Znfdhrenqr\Qrfxgbc\vrkcybere.rkr", "rot_13"))"
C:\Users\Masquerade\Desktop\iexplorer.exe
```

Decoding the ROT13 path stored by Windows UserAssist.

The executable was `iexplorer.exe`, with an extra `r`. Easy typo to miss, which was probably the whole idea.

ANSWER 3

OmniCTF{iexplorer.exe}

DEAD WEIGHT / PASTEBIN

The memory image pointed to the page

The browser and SmartScreen strings kept repeating Pastebin ID `egKYMmM4` . I found the normal page, raw endpoint and download endpoint in memory.



```
ayham@Ayham: ~/foren... x + v
ayham@Ayham:~/forensics/deadweight$ strings -a -n 4 DESKTOP-E0CHD27-20250604-195537.raw | rg -i
"pastebin.com/(raw/dl/)?egKYMmM4|Flag - Pastebin"
https://pastebin.com/egKYMmM4
https://pastebin.com/raw/egKYMmM4
https://pastebin.com/dl/egKYMmM4
Flag - Pastebin.com
ayham@Ayham:~/forensics/deadweight$ rg -n "CTF\{" pastebin_page.txt
I AM THE AUTHOR SO THE FLAG IS : CTF{Masquerade_IS_drunk}
```

Searching the memory strings for the repeated Pastebin ID.

The first flag was in a comment on the normal Pastebin page, not inside the raw paste. That's why searching the dump for the full flag returned nothing.

ANSWER 4`CTF{Masquerade_IS_drunk}`

The raw paste also had a two-DES verifier for the last answer.

DEAD WEIGHT / TWO DES

Meet in the middle and verify the hash

The paste gave a known plaintext and ciphertext pair. Instead of brute forcing both keys together, I encrypted from one side, decrypted from the other and matched the middle value.

```
ayham@Ayham: ~/foren... x + v
ayham@Ayham:~/forensics/deadweight$ python3 mitm.py
known plaintext : supersaferight?
known ciphertext: 57bd461497e572b0c5ec06c12dled8ce
middle value matched
ctf{a4062225bc93279ef48b44c6710079649a36b66755c56fc0dd7e7098dd266828}
ayham@Ayham:~/forensics/deadweight$ printf "%s" "ctf{a4062225bc93279ef48b44c6710079649a36b66755c56fc0dd7e7098dd266828}" | md5sum
b029aefa19e1889303614610be7d3295 -
```

The recovered candidate and the MD5 check from the challenge hint.

FORWARD

Encrypt with key1
candidates

BACKWARD

Decrypt with key2
candidates

CHECK

Known pair and MD5

ANSWER 5

ctf{a4062225bc93279ef48b44c6710079649a36b66755c56fc0dd7e7098dd266828}

The candidate matched the known DES pair and MD5 `b029aefa19e1889303614610be7d3295`, so that was the final value.